

A Replicated Data Types (RDTs)

A *Replicated Data Type (RDT)* is a class of distributed objects that comprises a *data type* and a set of *transactions* (or method calls) [23]. Clients of an object can request transactions at any replica, and replicas coordinate them. Transactions may not necessarily execute in the same order across replicas. Thus, the states of replicas may *diverge* if replicas execute certain transactions in different orders. A *replicated execution* is *convergent* if all replicas reach the same state after all transactions are propagated and executed at all replicas. An RDT may include an *integrity or consistency invariant* (as *C* in *ACID*) [10, 11, 102], a predicate on its data that must be maintained between transactions. Examples of integrity properties include requiring that a bank account’s balance is always non-negative, or referential integrity in relational schema, e.g., only a student registered at a university may enroll in a course.

Every transaction can be checked locally to be *permissible*, i.e., preserve integrity at its issuing replica. We refer to this local invariant/precondition validation performed before executing a transaction as a *permissibility check*. However, when transactions are issued concurrently at different replicas and then propagated, they can become impermissible, i.e., violate integrity in the target replicas. For example, suppose that two processes execute locally permissible withdrawal transactions from a bank account that ensures a positive balance; when propagated, the sum of the two withdrawals can overdraw the account. We say that a pair of transactions *convergence-conflict* and *integrity-conflict* if reordering them violates convergence and integrity, respectively. If either holds, we say that the pair conflicts. A transaction is conflict-free for convergence or integrity if there is no convergence or integrity-conflict with any other transaction, respectively. If both hold, we say that the transaction is conflict-free. An object is conflict-free if all its transactions are conflict-free. In order to preserve integrity, a transaction may be *dependent* on preceding transactions in the originating replica. For example, a bank account withdrawal may depend on a preceding deposit to ensure a positive account balance. In order to preserve integrity, a remote replica must execute transactions only after transactions it depends on are already implemented. A transaction is dependence-free if it does not depend on any other transaction.

RDTs are implemented using a *consistency model*. *Strong consistency* (also known as State Machine Replication (SMR)). ensures convergence and integrity by imposing a linear ordering of transactions across replicas; this, however, limits responsiveness and availability [1, 2, 21, 22, 28, 59, 80, 98], which inhibits practical deployments [25, 29, 48]. *Relaxed consistency* models [74, 91] are more performant than strong consistency [61, 81], but guarantee convergence or integrity properties for a limited subset of RDTs. For example, *Convergent Replicated Data Types (CRDTs)* [58, 93] are convergence conflict-free RDTs, and can be implemented using relaxed consistency. CRDTs have been successfully used as building blocks for distributed applications, including distributed databases [58], key-value stores [36], online data stores [94], and collaborative editing [70]. We consider *operation-based* CRDTs. Table B.1 lists six CRDTs including register, and different types of counters and sets, which serve as our benchmarks; the Supplement describes each CRDT in greater detail. For example, increment and decrement on the counter type are conflict-free for convergence. We note that

RDTs include a *query()* transaction, which reads values from the local object state; for brevity, we omit discussion of *query()*, except when needed.

Well-coordinated Replicated Data Types (WRDTs) [46, 47, 69] preserve both convergence and integrity, and are implemented using a hybrid consistency model [12–14, 34, 43, 63, 66–68, 96]: conflict-free transactions are implemented using relaxed consistency, and conflicting transactions are implemented using strong consistency. We will explain how WRDTs operate below. We note that WRDTs ensure convergence and integrity properties, whereas CRDTs only provide the former; CRDTs are a special case of WRDTs whose integrity predicate is the trivial assertion *true*.

WRDTs divide transactions into three mutually exclusive categories with increasing coordination cost: *reducible*, *irreducible*, and *conflicting* transactions. The first two are conflict-free. Reducible transactions are conflict-free, dependence-free, and summarizable. A transaction is *summarizable* if a sequence of invocations of that transaction can be aggregated into a single transaction. Summarizable transactions can be implemented efficiently: they can be aggregated at the local replica, and then a single transaction can be invoked at each remote replica to propagate the aggregated transaction. For example, the G-Counter contains a single non-negative value, *cnt*, initialized to zero. Each transaction *Increment(x)* adds $x \geq 0$ to *cnt*; repeated transactions can be summed locally, and then the aggregate *Increment(cnt)* is propagated once to each remote replica. Irreducible conflict-free transactions are conflict-free but are either not dependence-free or not summarizable.

Conflicting transactions require strong consistency to ensure the same order of execution across all replicas. A set of transactions that conflict with each other should synchronize their orders; hence, they are called a synchronization group. Table C.1 (SG) lists the synchronization groups for each WRDT. As an example, consider the *Movie* WRDT for a movie theater system. *Movie* provides the transactions *addMovie*, *deleteMovie*, *addCustomer*, and *deleteCustomer*, which add and delete movies from a theater’s database, and reserve customer seats for movie showings. (The transactions *deleteMovie* and *addCustomer* succeed only if the given movie exists.) Since add and remove operations to a set convergence-conflict, the first two transactions are a synchronization group, and the other two transactions are another synchronization group. We will describe the State Machine Replication (SMR) implementation in subsection 4.4. Transactions of a synchronization group use the same instance of SMR [89].

Table C.1 lists five WRDTs that we adopted [34, 47] to serve as benchmarks in our experiments.

B Conflict-free Replicated Data Types

A CRDT is a data type (or data structure) that can be replicated across in a distributed computer system in manner that provides several properties. Updates can be performed independently and concurrently at any replica; algorithms within the CRDT that are transparent to the user/programmer propagate updates to all other replicas, resolving any inconsistencies that may occur; and convergence under the strong eventual consistency model is guaranteed. It is important to note that strong eventual consistency *does not* guarantee any timeline by which convergence will be achieved.

Table B.1: CRDTs

CRDT	State	Reducible	Irreducible
G-Counter	Scalar C	<i>increment</i>	
PN-Counter	Scalar C	<i>increment</i> <i>decrement</i>	
LWW-Register	Scalar R Scalar T	<i>assign</i>	
G-Set	Set S	<i>insert</i>	
PN-Set	Array C		<i>insert</i> <i>remove</i>
2P-Set	Set A Set R		<i>insert</i> <i>remove</i>

Two different CRDT design and implementation strategies have been proposed: operation-[17, 62] and state-based [6, 18]. Both operation-based and state-based CRDTs provide strong eventual consistency, and they are theoretically equivalent, meaning that any operation-based CRDT can emulate its state-based counterpart, and vice-versa [92]. State-based CRDTs require periodic transmission of each replica’s state to other replicas, requiring a non-trivial state merge operation, depending on the level of sophistication of the CRDT. In contrast, operation-based CRDTs require each replica to broadcast each method invocation to the other replicas, which increases network communication. The CRDT Wikipedia page³ provides further details.

B.1 CRDT Benchmark Summary

The CRDTs used for experimental evaluation are described as follows. Our evaluation is limited to scalar instances of each CRDT.

G-Counter⁴: A grow-only counter, which can only be incremented.

PN-Counter: A positive-negative counter which supports increment and decrement operations. A PN-Counter comprises two G-Counters: one for increments, and one for decrements.

LWW-Register: A last writer wins register, which supports assignment of values to the register. Unique timestamps are associated with each assignment, which ensures a total order of operations. The register always retains the most recently written value.

G-Set: A grow-only set which supports insertion, but not removal, of set elements.

PN-Set: A counter is associated with each element of a set: insertion increments the counter; removal decrements the counter. An element is present in the set if its counter is positive. A PN-set may represent a non-traditional multi-set, in which a positive counter value represents the number of occurrences of an element in the set; however, when the counter becomes negative, multiple insertion operations are needed to reintroduce the element to the multi-set.

2P-Set: A two-phase set that supports insertion and removal of set elements. Once removed, an element cannot be reinserted into the set. A 2P-Set is typically implemented using two G-Sets.

³https://en.wikipedia.org/wiki/Conflict-free_replicated_data_type

⁴We do not include G-Counter as a benchmark; it is a building block for the PN-Counter.

C Well-Coordinted Replicated Data Types

WRDTs generalize CRDTs with the addition of conflicting methods which require strong consistency [46, 47, 69]. Strong consistency necessitates an SMR protocol to synchronize conflicting calls. Like CRDTs, WRDTs provide strong eventual consistency for non-conflicting method categories. WRDTs thereby guarantee convergence for both conflicting and non-conflicting method categories. WRDTs also provide invariants which are satisfied through permissibility checks, thereby guaranteeing integrity.

C.1 WRDT Benchmark Summary

The WRDTs used for experimental evaluation are described as follows.

Bank Account: A distributed bank account that maintains a scalar balance; *deposit()* and *withdraw()* methods respectively increase and reduce the balance. Withdrawals that cause overdrafts (funds less than 0) are impermissible.

Courseware: A university registrar that manages courses to be taught (*addCourse()*, *deleteCourse()*), enrolls students in the University (*addStudent()*), and enrolls students in courses (*enroll()*).

Project: Business software that manages projects (*addProject()*, *deleteProject()*), enrolls personnel (*addEmployee()*) and assigns personnel to work on projects (*assign()*).

Movie: A database of movies (*addMovie()*, *deleteMovie()*) and customers (*addCustomer()*, *deleteCustomer()*).

Auction: An e-commerce site that manages users (*registerUser()*), lists items for direct sale (*sellItem()*), allows users to purchase items (*buyItem()*), and facilitates item sales via auction (*openAuction()*, *placeBid()*, *closeAuction()*).

Bank Account’s state comprises a single scalar, i.e., one account with one balance. The other WRDTs’ states comprise two or three sets, and in the case of Auction, three sets and one array.

D RDMA

RDMA was originally proposed for high-performance computing (HPC) clusters through the Infiniband protocol, and was later adapted for data center networks as RDMA over Converged Ethernet (RoCE), a link-layer protocol; RoCEv2 is implemented on top of UDP/IPv4 or UDP/IPv6 as an internet-layer protocol, which supports packet routing. Further details can be found on the RoCE Wikipedia page⁵.

D.1 RDMA vs. Socket-based Networking

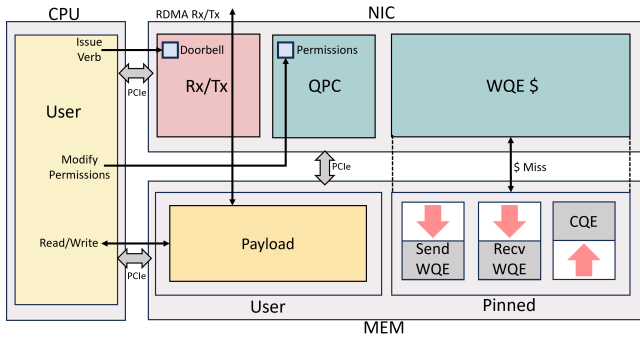
Figure 15 respectively depicts the components (CPU, memory, NIC) that implement socket-based and RDMA networks; both figures assume that all three components are connected via PCIe.

In Figure 15, the operating system (OS), but not the user-space application, communicates with the NIC. User-space applications must incur the overhead of system calls to transmit and receive data. All data received by the NIC is written to memory managed

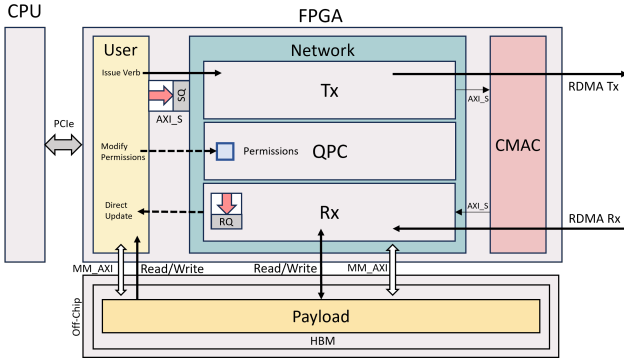
⁵https://en.wikipedia.org/wiki/RDMA_over_Converged_Ethernet

Table C.1: WRDT Methods and Invariants

RDT	State	Reducible	Irreducible	Conflicting	Sync Group
Bank Account	Scalar B	$deposit(d)$		$withdraw(w)$ where $B - w \geq 0$	1
Courseware	Set S Set C Set E		$addStudent(s)$ where $s \notin S$	$addCourse(c)$ where $c \notin C$ $deleteCourse(c)$ where $c \in C$ $enroll(s, c)$ where $s \in S, c \in C, \langle s, c \rangle \notin E$	1
Project	Set E Set P Set A		$addEmployee(e)$ where $e \notin E$	$addProject(p)$ where $p \notin P$ $deleteProject(p)$ where $p \in P$ $assign(e, p)$ where $e \in E, p \in P, \langle e, p \rangle \notin A$	1
Movie	Set C Set M			$addCustomer(c)$ where $c \notin C$ $deleteCustomer(c)$ where $c \in C$ $addMovie(m)$ where $m \notin M$ $deleteMovie(m)$ where $m \in M$	2
Auction	Set U Set A Set I Array $S[]$	$sellItem(i, u)$	$openAuction(a)$ where $a \notin A$	$registerUser(u)$ where $u \notin U$ $buyItem(i, u)$ where $i \in I, S[i] \geq 1, u \in U$ $placeBid(a, bid, u)$ where $a \in A, u \in U$ $closeAuction(a)$ where $a \in A$	3



(a) Architectural components that implement RDMA for a traditional CPU-based network.



(b) RDMA architecture for a network-attached FPGA featuring a soft RNIC [95].

Figure 15: Architectural components that implement RDMA for both a CPU and FPGA.

by the OS kernel; once received, the OS copies data to user-space memory, before the user-space application can access the data.

In Figure 15, the user-space application communicates directly with an RDMA-enabled NIC (RNIC), bypassing the OS kernel. As the RNIC provides remote access to user-space memory, the RNIC's Queue Pair Context (QPC) assumes responsibility for managing memory access permissions, a function typically performed by the

OS in other contexts. The three queues that comprise an RDMA queue-pair (QP) for each RDMA connection are allocated in a pinned memory segment. The RNIC contains a cache that provides fast access to QPs. Data is remotely written to user-space memory, eliminating the need to copy data from kernel-managed into user-space memory in Figure 15. User-space applications that expect to receive data poll their memory segments to access data written by remotely-initiated **RDMA_Writes**, and their QPs to detect completion of locally-initiated **RDMA_Reads**.

To refresh terminology, the QP on each side of a connection comprises three queues: a Send Queue (SQ), a Receive Queue (RQ), and a Completion Queue (CQ). The user-space application issues a work request by posting a Work Queue Entry (WQE) to the QP: a Send Queue Entry (SQE) is posted to the SQ, a Receive Queue Entry (RQE) is posted to the RQ, and a Completion Queue Entry (CQE) is posted to the CQ.

D.2 RDMA_READ

Figure 16 depicts the **RDMA_Read** path for a CPU-based RDMA network. The user-application (1) posts a *SQE* and *RQE* to the *SQ* and *RQ* and (2) triggers the RNIC by writing to the doorbell register; both of these steps entail PCIe transactions and the first step accesses memory. The RNIC (3) fetches the *SQE* from memory, (4) checks the QP state, (5) verifies permissions, and (6) packages the **RDMA_Read** operation for transmission over the network; step (4) incurs PCIe transaction and memory access latencies. On the receiving side, the RNIC (7) retrieves and unpacks that **RDMA_Write** operation, (8) verifies permissions, and (9) reads the payload from memory over PCIe at the address specified in the operation. The receiver's RNIC (a, b) returns the payload over the network to the sender who (c) writes the received payload to memory, (d) fetches the *RQE* from the *RQ* and (e) writes a *CQE* to the *CQ*, signaling completion of the operation; steps (c), (d), and (e) incur PCIe transaction and memory access latencies. The sender's user application can (f) access memory at any time to retrieve the remotely read payload.

D.3 RDMA_WRITE

Figure 17 depicts the **RDMA_Write** path for a CPU-based RDMA network. The user-application (1) writes the payload intended for transmission into memory; (2) posts an *SQE* to the *SQ*; and (3) triggers the RNIC by writing to a doorbell register; all three of these

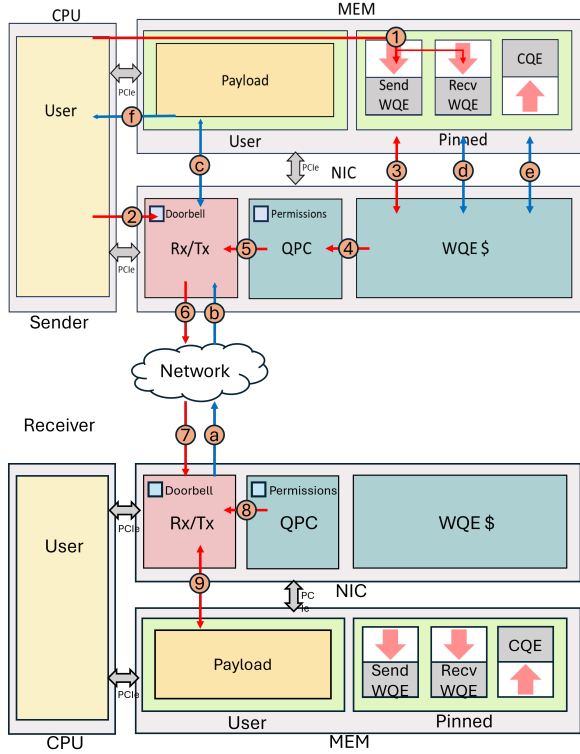


Figure 16: CPU Nic read path

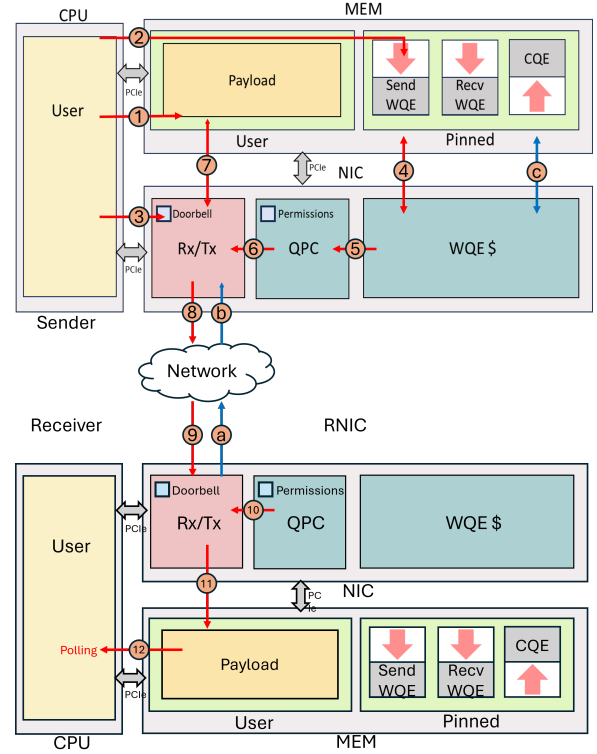


Figure 17: CPU Nic write path

steps entail PCIe transactions, and the first two access memory. The RNIC (4) fetches the *SQE* from memory, (5) checks the *QPC* state, (6) verifies permissions and packet header information, (7) retrieves the payload from memory, and (8) packages the **RDMA_Write** operation for transmission across the network; steps (4) and (7) incur PCIe transactions and memory access latencies. On the receiving side, the RNIC (9) retrieves and unpacks the **RDMA_Write** operation, (10) verifies permissions, and (11) writes the payload to memory over PCIe. The receiver's RNIC (a, b) returns an acknowledgment (ACK) to the sender, whose RNIC completes the **RDMA_Write** operation by posting a *CQE* to the *CQ*. The receiver's user application can (12) access memory at any time to retrieve the remotely written payload.

D.4 RDMA_READ (FPGA Implementation)

Figure 18 depicts the **RDMA_Read** path for a pair of network-attached FPGAs.

The user kernel (1) pushes the **RDMA_Read** operation to the *SQ*, implemented as a AXI Stream interface. (2) The Network kernel pops the verb from the *SQ* and (3) checks the *QPC* for permissions, and connection information. Since the operation is **RDMA_Read**, (4) an *RQE* is posted to the *RQ*, which resides in on-chip BRAM. The RDMA packet is (5) packaged and transferred to the CMAC through an AXI Stream. The CMAC takes the RDMA packet, adds Ethernet headers, and (6, 7) transmits the packet over the network. At the receiving node, the CMAC strips the Ethernet header and

(8) passes the RDMA packet to the Network kernel. The remote Network kernel (9) checks permissions, and (10) issues the read operation to memory through a memory-mapped AXI interface to obtain the payload, and (a) passes the payload to the network kernel for transmission back to the node that initiated the verb. The RDMA packet is (b, c, d) returned and (e) received by the Network kernel. The Network kernel (f) pops the *RQE* from the *RQ* and writes the payload to the requested address in memory. The User kernel (g) accesses the payload through a memory-mapped AXI interface.

D.5 RDMA_WRITE (FPGA Implementation)

Figure 19 depicts the **RDMA_Write** path for a pair of network-attached FPGAs.

The user kernel (1) pushes the **RDMA_Write** verb to the *SQ*, implemented as a AXI Stream interface. (2) The Network kernel pops the verb from the *SQ* and (3) checks the *QPC* for permissions, and connection information. The Network kernel maintains an queue of expected acknowledgments (ACKs) for retransmission purposes; before transmitting an RDMA packet, the Network kernel posts an expected ACK into the ACK queue. The RDMA packet is (5) packaged and transferred to the CMAC through an AXI Stream. The CMAC takes the RDMA packet, adds Ethernet headers, and (6, 7) transmits the packet over the network. At the receiving node, the CMAC strips the Ethernet header and (8) passes the RDMA packet to the Network kernel. The remote Network kernel (9) checks permissions, and (10) writes the payload to memory

Table D.1: Average latencies measured for several RDMA verbs that write to FPGA integrated storage.

Operation	Latency
Write	413 <i>ns</i>
BRAM_Write	309 <i>ns</i>
BRAM_Write_Through	309 <i>ns</i>
Register_Write	285 <i>ns</i>
Register_Write_Through	285 <i>ns</i>

not report the time to generate and receive ACKs. We identify one RDT use-case for a write-through verb (subsection 4.3).

E Extended Related Work

Positioning and scope. We use this claim narrowly: SafarDB is not the first FPGA-based replicated storage system, nor the first RDMA-based replication system. SafarDB builds on four existing lines of work: StRoM’s FPGA RoCE/RDMA stack [95], Mu’s RDMA-based SMR protocol [3], Hamsaz/Hamand’s RDT and WRDT semantics [46, 47], and FPGA or programmable-network replication systems such as CBox, Caribou, Waverunner, P4xos, and NetPaxos [5, 32, 33, 50, 51]. SafarDB’s contribution is to co-locate RDMA communication, RDT execution, and the strong-consistency control path inside a network-attached FPGA, and to add the custom RDMA/RPC mechanism, direct permission switching through RNIC state, an FPGA implementation of the adopted hybrid relaxed-consistency/strong-consistency RDT model, and hybrid FPGA/host memory mode.

RDTs and coordination avoidance. Prior work on CRDTs, RDTs, and coordination avoidance provides the semantic foundation for SafarDB. CRDTs provide convergence for commutative operations [23, 93], while systems and analyses for highly available or weakly consistent databases identify when coordination is required to preserve application invariants [10, 15, 16, 35]. Hamsaz and Hamand specialize this direction for WRDTs and RDMA-based replicated execution [46, 47]. SafarDB differs from these systems by changing the execution substrate: instead of executing the RDT logic on CPUs over RDMA, it moves the RDT operation handlers, RDMA endpoint, and strong-ordering support onto the FPGA.

Strongly ordered replication and SMR. SafarDB targets a different point in this design space: FPGA-resident execution of RDT/WRDT operations with both relaxed and strongly ordered replication paths. Consensus/SMR work relates to SafarDB’s support for strongly ordered operations. SMR protocols, which are needed for WRDT conflicting transactions, have been designed for message-passing [29, 48, 60, 73, 77, 79] and RDMA [3, 4, 42, 52, 54, 55, 72, 82, 97, 99, 103] networks. In terms of hardware acceleration, HovercRaft [56] and Waverunner [5] accelerate Raft [79] using an in-network programmable P4 ASIC and an FPGA-based Soft NIC, respectively. Consensus-in-a-Box (CBox) [51] implements the full Zookeeper Atomic Broadcast (ZAB) protocol [48, 73], including leader election, failure recovery, and application code on the FPGA while storing the replication log in onboard DRAM. We opted to implement Mu [3] on the FPGA as our RDT implementations were RDMA-based; non-RDMA-based RDTs could be ported to network-attached FPGAs using either Waverunner [5] or CBox [51] to provide consensus.

FPGA storage and network-accelerated replication. Caribou [50] is particularly relevant to SafarDB’s FPGA-only configuration because it also places replicated storage on network-attached FPGAs. Caribou exposes a replicated key-value storage interface over regular TCP/IP sockets, stores key-value data in FPGA-attached memory, performs near-data filtering/scans in the FPGA, and replicates writes through a ZAB-based protocol over the same network. SafarDB differs in both the communication and replication abstractions: it uses an RDMA/RoCE-style network endpoint rather than a TCP/IP key-value interface; it executes application-level RDT/WRDT handlers, including key-value workloads such as YCSB, rather than exposing only a key-value storage API; it allows non-conflicting RDT operations to bypass strong ordering; and it accelerates the strong ordering path using FPGA-resident SMR and direct RNIC-state access.

Waverunner [5] is also closely related because it accelerates state-machine replication using an FPGA-based SmartNIC and evaluates replicated services, including key-value workloads. SafarDB differs from Waverunner in where application execution and consistency selection occur. Waverunner keeps the replicated application on the host and uses the FPGA primarily to accelerate the Raft replication path. As a result, Waverunner accelerates the common strongly ordered Raft replication path, while leaving application logic in software and not providing FPGA-resident RDT execution or RDT-aware consistency selection. SafarDB places RDT/WRDT operation handlers and the RDMA-capable endpoint inside the network-attached FPGA, allowing non-conflicting operations to be executed on the relaxed path and conflicting operations to be strongly ordered and executed on FPGA-resident SMR.

P4xos and NetPaxos also accelerate consensus using programmable network hardware [32, 33]. They focus on moving Paxos roles or ordering logic into switches or programmable forwarding devices, whereas SafarDB keeps the replicated application state and RDT execution at the FPGA-attached storage endpoint. Thus, those systems are complementary consensus accelerators; they do not provide RDMA-based RDT execution, hybrid relaxed-consistency/strong-consistency replication paths, or FPGA/host memory placement for replicated data. We therefore treat NetPaxos as a programmable-network consensus accelerator rather than as an experimental storage/RDT baseline.

RDMA/RPC communication primitives. SafarDB relates to prior work that extends RDMA/RPC and programmable-NIC interfaces. For example, Prism [24] introduces new primitives to support common remote access patterns, StRoM [95] and RMC [8] provide RDMA or RDMA-like extensions for remote computation and RPC-style execution, and KV-Direct [65] uses programmable NICs to extend RDMA-style access for direct key-value operations. The nanoPU provides an RDMA-inspired fast path between the NIC and CPU register file to support μ s-scale RPCs [49]. Either Hamand’s or SafarDB’s RDTs may further benefit from Prism’s primitives or nanoPU acceleration. SafarDB’s SMR could similarly complement KV-Direct-style programmable-NIC key-value access in a replicated key-value store.

This line of work also highlights a design trade-off between RDMA and RPC abstractions. One-sided RDMA avoids remote CPU involvement, but it exposes memory operations rather than

application-level execution. Two-sided RDMA SEND/RECV provides request delivery, but still requires receive queues and dispatch logic at the target. RPC-style NIC extensions instead allow a network request to directly invoke application-specific handlers. SafarDB follows this last direction, but specializes it for FPGA-resident RDT execution: incoming operations can both move data and trigger the appropriate RDT or SMR logic without routing the common case through host memory.

FPGA database query processing. Prior FPGA database systems accelerate event processing, multi-query stream processing, and flexible query execution on reconfigurable hardware [75, 76, 86, 87]. These systems show how database query-processing operators can be optimized on FPGAs. SafarDB targets a different layer of the data-management stack: replicated transactional processing, where the main challenge is coordinating RDT/WRDT operations across replicas using relaxed and strongly ordered replication paths.

SmartNIC architectures. SmartNICs [26, 39–41, 84] integrate programmable accelerators, allowing servers to offload tasks relating to networking to the network interface. *On-path* (“bump in the wire”) SmartNICs place programmable elements on the packet processing path, which adds latency when they are not utilized, while *off-path* SmartNICs [71, 90, 101] integrate a network switch, which removes the programmable elements from the packet processing path, but adds a small amount of routing latency. Prior work has noted a lack of versatility in SmartNIC functionality, as SmartNICs are often limited to specific tasks such as RDMA [45]. Unlike SmartNICs, SafarDB’s network stack is implemented within the FPGA; while less efficient than using an ASIC, SafarDB eliminates inter-chip communication overheads that are inherent to SmartNICs while enabling application developers to design and deploy application- or domain-specific on-path, off-path, or hybrid communication architectures.

Therefore, SafarDB should be viewed as a network-attached FPGA replication engine rather than as a conventional CPU-hosted database with an offloaded NIC function. The relevant distinction is that the RDMA-capable endpoint, replication control path, and RDT operators are co-designed inside the same FPGA, which lets SafarDB remove PCIe and host-memory interactions from the common FPGA-resident execution path.

F Additional Experimental Results

F.1 Hardware Comparison for Cross-Cluster Evaluation

F.2 Latency breakdown

Figure 22 reports the latency breakdown of computation versus communication for conflicting transactions. In this experiment, computation refers to the latency of modifying local data for the given workload, while communication refers to the latency of coordinating conflicting transactions. Most of the time in both systems is spent coordinating the transaction, with computation contributing only minimal overhead. SafarDB’s computation is 4x faster than Hamband’s, while its communication is 2x faster. SafarDB leverages FPGA hardware acceleration to perform both computation and coordination; the FPGA’s RDMA implementation allows coordination to be completed efficiently.

Table F.1: Hardware comparison between OCT (SafarDB) and ACES (Hamband). Every dimension except the FPGA accelerator favors Hamband, making the cross-cluster comparison conservative with respect to SafarDB’s reported gains. Intel’s specification sheets report memory speed using different units for the two CPUs.

Dimension	OCT (SafarDB)	ACES (Hamband)
CPU model	Xeon Gold 6226R [30]	Xeon Platinum 8468 [31]
CPU generation	Cascade Lake (2020)	Sapphire Rapids (2023)
Cores / threads	16 / 32	48 / 96
Base / turbo MHz	2900 / 3900	2100 / 3800
Memory type	192GB DDR4	512GB DDR5-4800
L3 cache	22 MB	105 MB
Network bandwidth	100 Gbps (GbE)	200 Gbps (NDR IB)
Switch fabric	100GbE Ethernet	NDR400 InfiniBand
RDMA NIC	Soft RNIC on FPGA	ConnectX-7 (hardened)
FPGA accelerator	Alveo U280 (8 GB HBM)	None
PCIe Generation	PCIe 3.0	PCIe 5.0
Memory speed	2933 MHz	4800 MT/s

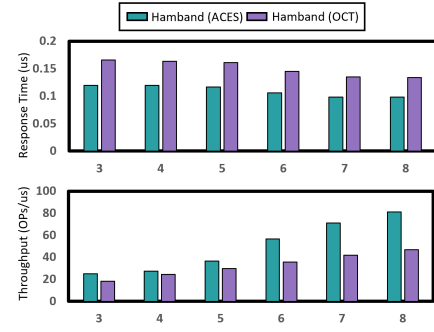


Figure 21: Performance of Hamband running SmallBank on OCT vs ACES.

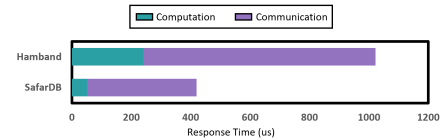


Figure 22: Latency breakdown of Hamband and SafarDB.

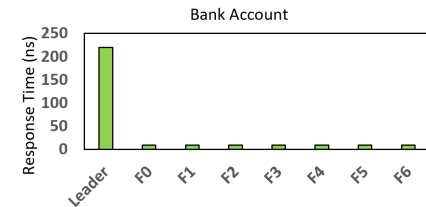


Figure 23: Execution time of all replicas for Bank Account WRDT in a 15% write percentage 8-Node configuration

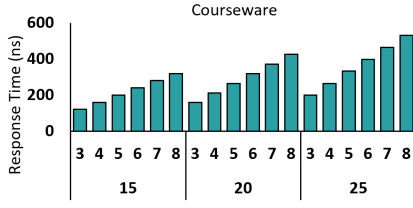


Figure 24: Execution time of leaders in Courseware WRDT

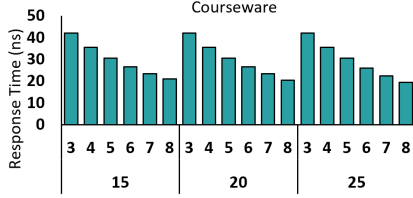


Figure 25: Execution time of followers in Courseware WRDT

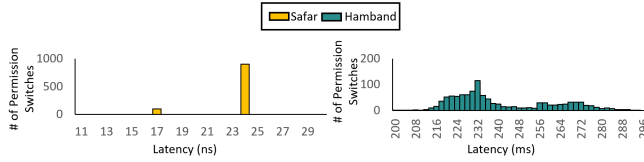


Figure 26: Permission Switch histograms

F.3 Leader and Follower Execution Breakdown

Figure 23 reports the execution time of the Bank Account WRDT with 8 replicas and a write percentage of 15%. The Leader’s execution time is more than twice as larger as the execution times of the seven follower replicas F0-F6; the follower execution times are approximately equal. Throughput is defined as the total execution time of the workload divided by the total execution time of the system. The total execution time is constrained by the longest-running replica, which is the Leader in Figure 23. Presuming that Bank Account is representative of all WRDTs, Figure 23 indicates that the most promising way to improve throughput is to improve the performance of the Leader, which means focusing on how conflicting transactions and coordination are implemented.

Figure 24 reports the leader execution times across 3-8 replicas and write percentages of 15%, 20%, and 25%, for the Courseware WRDT. Increasing the write percentage tends to increase leader execution time, since there are more conflicting transactions. To recall, the write percentage is the fraction of methods that write state, which includes reducible, irreducible conflict-free, and conflicting methods, whereas all remaining calls invoke the *query()* method. Increasing the number of replicas tend to increase leader execution time, due to the increased cost of coordinating a larger number of followers.

Conversely, Figure 25 reports the average execution time of all followers across the same range of replicas and write percentages.

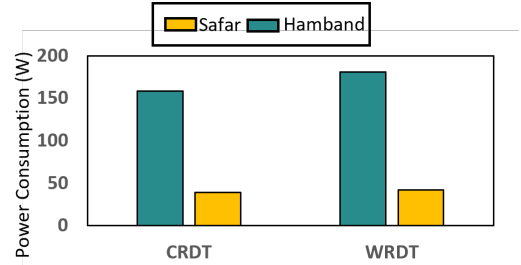


Figure 27: Power Consumption

As the number of replicas increase, each follower invokes fewer calls, which reduces execution time. Increasing the write percentage marginally increases execution by adding more conflict-free transactions to execute; as conflict-free methods lack coordination, the slow down is minimal. Conflicting calls do not impact follower execution time to the same extent that they impact leader execution time.

F.4 Permission Switch Histograms

F.5 Power Consumption

FPGA power consumption was measured following AMD Xilinx Documentation [7]. CPU and I/O power consumption was measured with a combination of PowerTop [83] and Scaphandre [88].

Figure 27 reports the peak power consumption of CRDTs and WRDTs averaged across use cases and write percentages. SafarDB consumes about 35W, while Hamband consumes about 160W, so SafarDB consumes approximately 4.5 $\times$ less power than Hamband while providing improved performance. SafarDB’s power consumption includes the entire Alveo U280 card and HBM, while Hamband’s includes the full system with CPU, RNIC, memory, and I/O. For Hamband, approximately two-thirds of the power consumption comes from the CPU, and one-third comes from I/O, including memory, the RDMA NIC, and the PCIe interface. This disparity is expected for FPGAs because they operate at lower clock frequencies,

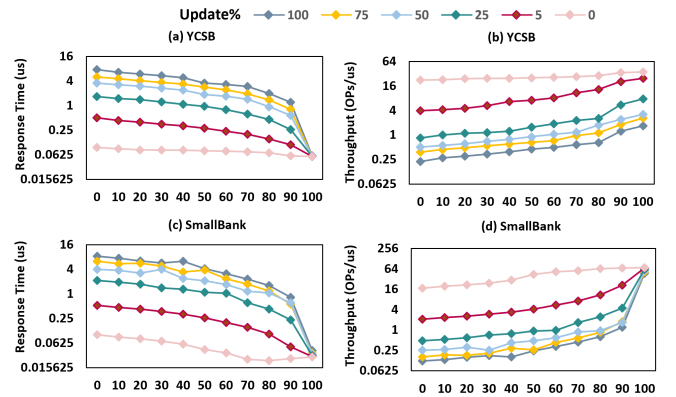


Figure 28: Response time and throughput for YCSB/Small-Bank with different percentages of OPs assigned to the FPGA.

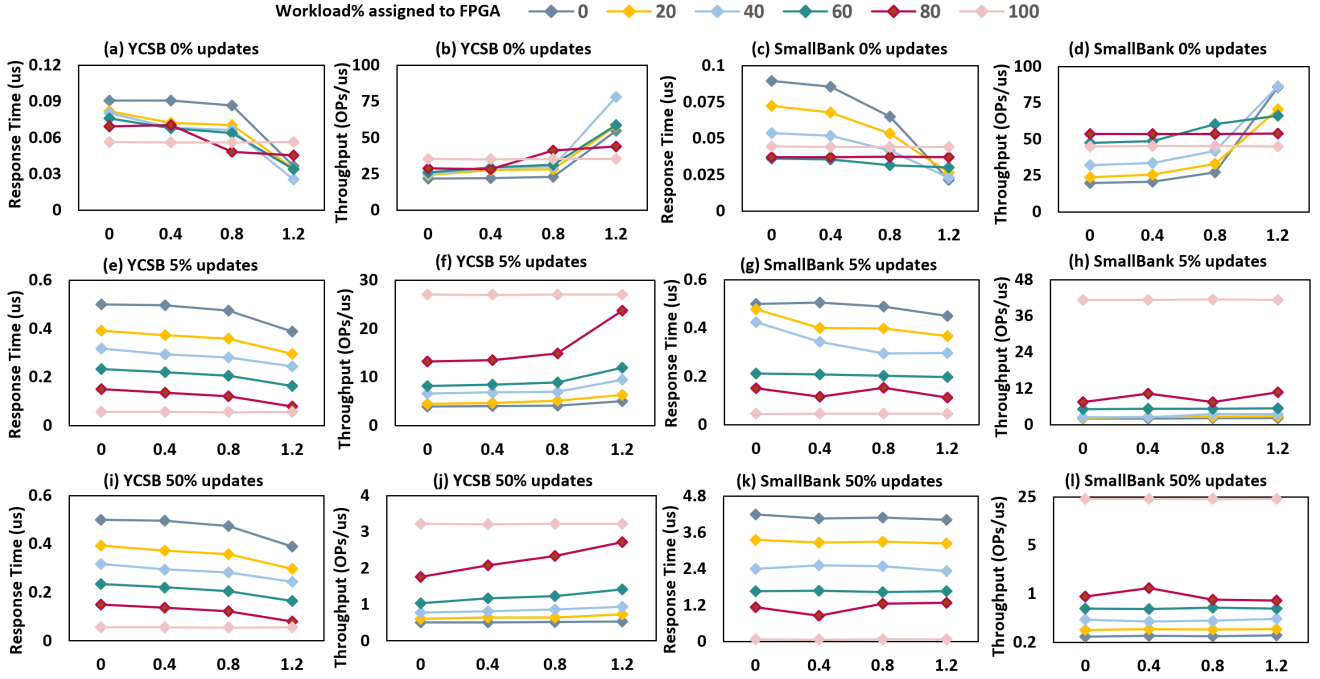


Figure 29: Response time and throughput for YCSB and SmallBank with different θ values of Zipfian.

avoid CPU-style instruction fetch/decode overheads, and reduce data movement through the cache hierarchy [57].

F.6 Hybrid-Mode Detailed Results

Figure 28 and Figure 29 provide the detailed hybrid-mode results summarized in the Evaluation section. Figure 28 varies the fraction of YCSB and SmallBank requests that target FPGA-resident keys/accounts versus host-resident keys/accounts. These results show the capacity/performance trade-off of hybrid mode: host memory expands the dataset beyond FPGA memory, while response time decreases and throughput increases as more requests are served by FPGA-resident state. Figure 29 varies the Zipfian skew and shows that skew can improve host-side accesses through CPU cache locality, but this benefit becomes smaller as the update ratio increases or as more requests already target FPGA-resident state.

The percentage-assignment experiment separates capacity from placement. The workload remains the same, but the fraction of keys/accounts served from FPGA memory increases along the x-axis. The 100% point corresponds to the FPGA-only case, while lower percentages model deployments where only part of the replicated state is placed in FPGA memory and the remaining state stays in host memory. This explains why response time improves most consistently as the FPGA-resident fraction grows: fewer requests cross the FPGA-host boundary, and more operations complete in the FPGA-resident RDT handlers.

The Zipfian experiment exercises a different effect. Higher θ concentrates requests on fewer records, which can improve CPU-cache locality for host-resident state. This benefit is most visible

when a substantial fraction of requests still accesses host memory. When most requests already target FPGA-resident state, or when the workload has more updates, the benefit from skew is smaller because replication, synchronization, and FPGA-side execution costs dominate. Together, these detailed plots support the main-text summary that hybrid mode is both a capacity mechanism and a placement knob for keeping frequently accessed state in FPGA memory while preserving one replication interface for both storage locations.